

An Effort-Aware Just-in-Time Software Defect Prediction Model based on LightGBM

Liqiong Chen

School of Computer Science and Information Engineering
Shanghai Institute Of Technology
Shanghai, China

Ying Wang

School of Science
Shanghai Institute Of Technology
Shanghai, China

Abstract—Effort-aware Just-in-Time Software Defect Prediction is a fine-grained technique and takes the cost of detection into account to detect more defective changes with limited testing resources. Since many existing models lack better performance, an effort-aware just-in-time software defect prediction model is proposed called EALGB based on LightGBM in this paper. Intensive simulation experiments have been conducted on six open-source datasets to demonstrate the performance of the proposed method. The prediction model is evaluated by two widely recognized metrics, ACC and Popt, in three scenarios called cross-validation, cross-project cross-validation and time-aware cross-validation. Empirical results demonstrate that the proposed algorithm EALGB has good prediction performance.

Keywords—Just-in-Time software defect prediction; Effort-aware; LightGBM ; Cross-validation

I. INTRODUCTION

Nowadays, software is used in every aspect of people's work and daily life. However, with the increasing size and complexity of software, defects are becoming a serious threat to software quality. Since developing flawless software products is impossible, it is important to identify software defects as early as possible, so that developers can focus more precisely on specific code ranges, and thus reduce the time and effort of their tests. For this reason, researchers have proposed change-level defect prediction called just-in-time software defect prediction (JIT-SDP) [1]. Just-in-time software defect prediction has the advantages of fine-grained, immediacy, and easy traceability over traditional software defect prediction [2]. Some other researchers have proposed Effort-aware software defect prediction by adding the cost of code inspection into consideration. Kamei et al. introduced Effort-aware technique to JIT-SDP for the first time and proposed the Effort-aware JIT-SDP [3]. However, the predictive performance of many existing models is not satisfactory.

Therefore, in this paper, we construct an effort-aware JIT-SDP model called EALGB with better prediction effect based on LightGBM. Our main contributions are shown below:

- The proposed model was applied in 10 times 10 fold cross-validation, cross-project validation, and time-aware cross-validation and evaluated by ACC and Popt.

- The proposed model is a supervised model based on LightGBM. The experimental results show that EALGB outperforms the remaining supervised and unsupervised models in almost all of the two metrics in terms of prediction results.

The rest of this paper is organized as follows. Section 2 describes the related work. Section 3 describes the proposed method in detail. The experimental analysis is described in Section 4. Section 5 concludes the full paper.

II. RELATED WORK

A. Just-in-time software defect prediction

Kamei et al. [1] firstly proposed the concept of just-in-time software defect prediction (JIT-SDP) for software quality assurance. With the application of deep learning in many fields, some researchers have utilized it to improve JIT-SDP performance. Ardimento et al. [4] introduced a deep spatiotemporal convolutional network variant based on layered attention layers to perform defect prediction. Gesi et al. [5] proposed a Siamese Network based JIT-SDP few shot learning framework (SIFTERJIT) in conjunction with DeepJIT. In addition, Duan et al. [6] demonstrated the significant impact of duplicate changes on the evaluation and interpretation of the JIT-SDP model by simulating duplicate changes in eight Apache open source projects. Zhao et al. [7] proposed a new compositional model IDL to perform JIT-SDP tasks, effectively alleviating the problem of data imbalance.

B. Effort-aware software defect prediction

In recent years, researchers [8] have shown that 80% of defects are contained in 20% of the code. Kamei et al. [1] initially integrated effort-aware into JIT-SDP. Considering the performance of sorting, a new method for measuring effort known as Multi-metric Joint Calculation (MMJC) was proposed by Chen et al. [9]. Zheng et al. [10] proposed a JIT-SDP model based on random forest classification, which uses six open-source projects from different fields to improve prediction efficiency. Cabral et al. [11] proposed a novel approach to JIT-SDP, resulting in improved and more consistent predictive performance. Moreover, Çarka et al. [12] explored the efficacy of Effort-aware JIT-SDP, ultimately proposing and assessing the standardization of PofB.

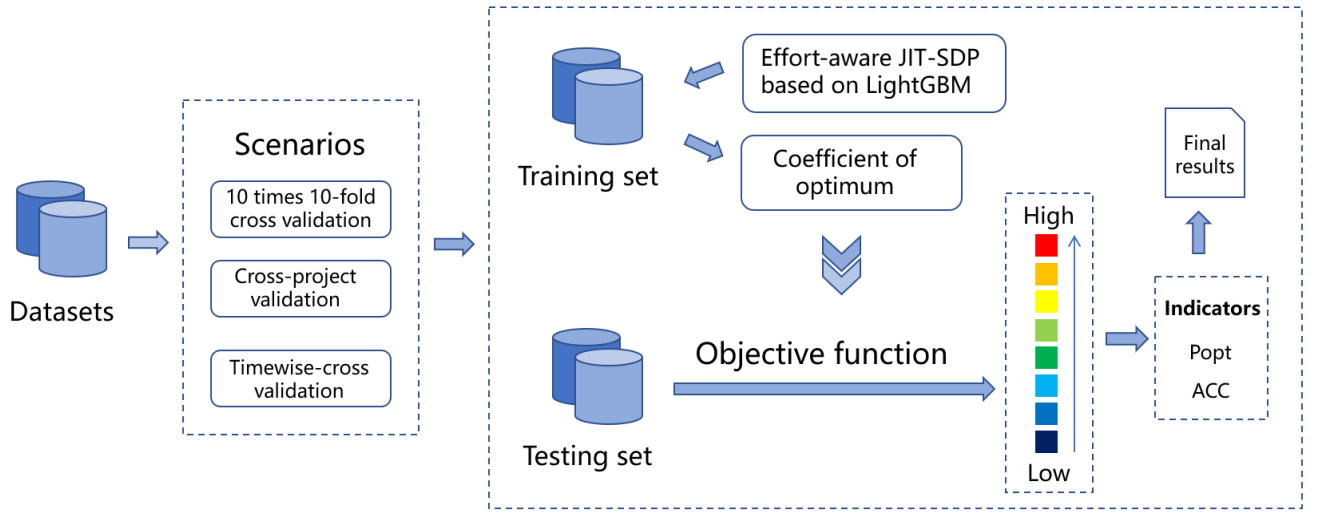


Fig.1. APPROCH REVIEW

III. APPROACH

A novel Effort-aware JIT-SDP model called EALGB is proposed. Fig. 1 depicts an overview of the proposed approach.

- First, defect data is extracted from the software history library.
- Identify the application scenarios for this experiment: 10 times 10-fold cross validation, cross-project validation and timewise-cross validation.
- Divide the training and test sets: to ensure that the model performs well in development and evaluation, we divide 80% of the data extracted in the first step into a training set and 20% into a test set, and test them in the three different scenarios mentioned above respectively.
- Experiments on the proposed model: the EALGB model is proposed and experiments are conducted on the extracted dataset to verify the validity of the model.

In this research, a code change is defined as $c = \langle m_1, m_2, \dots, m_n \rangle$, where m_i denotes the value of the metric element and n denotes the number of metric elements. If change c contains defects, then the label y of c is 1, otherwise it is 0. The defect *density* of c is defined as eq(1).

$$density = \frac{y}{churn}$$

(1)

where *churn* denotes the number of lines of code added and removed by change c . Suppose there are k code changes, $c_1, c_2, \dots, c_k$, which vary in ascending order of predicted defect density. Therefore, the sum of the defect densities is defined as eq(2).

$$D = \sum_i^K d_i$$

(2)

where d_i is the actual defect density of the i -th code change.

It has been shown that tree-based classifiers can achieve better performance in software defect prediction. LightGBM is a new integrated learning method developed by Microsoft Asia Institute. Compared with XGBoost, LightGBM has faster iteration speed and higher prediction accuracy. EALGB uses LightGBM to build a classifier, and then changes with a prediction of 1 are sorted in ascending order by the value of churn. Priority is given to detecting changes that may be defective and have a small churn value. The algorithmic flow of EALGB is shown in Algorithm 1.

Algorithm 1 Pseudocode for EALGB

```

1: CBS(LightGBM, TestSet)
2: Input:
3: LightGBM: The classifier built on training dataset
4: Test: Testing set of changes
5: Output:
6: RankedList: Ranked list of changes for inspection
7: Method:
8: Defective, NonDefective = ∅;
9: for all changes  $c \in \text{TestSet}$  do
10: Use LightGBM to predict the the label  $l$  of change  $c$ ;
11: if  $l$  is potentially defective then
12:   Add  $c$  into Defective;
13: else
14:   Add  $c$  into NonDefective;
15: end if
16: end for
17: Let RankedList = Changes in Defective sorted in
   Ascending order of size;
18: return RankedList;
    
```

IV. EXPERIMENTAL ANALYSIS

This section describes the configuration of the experiment in the following aspects: data sets, data preprocessing, validation scenarios, experimental design, and evaluation metrics. The hardware configuration for the experiments is: Intel(R) Xeon(R) Silver 4110 CPU @2.10GHz. The operating system is Windows 10. The development environment is Python 3.9.

A. Data Sets

The datasets [13] used in this paper are open source datasets from six different domains that are widely used in the field of instant software prediction. The six open source datasets are Bugzilla (BUG), Columba (COL), Eclipse JDT (JDT), Mozilla (MOZ), Platform (PLA), Postgres (POS). The basic information of the data is shown in Table 1, which contains the software development cycle, the number of defect changes and the defect rate.

TABLE I. THE BASIC INFORMATION OF THE DATA

Project	Period	Changes	Defects	Defect Rate(%)
BUG	1998/8-2006/12	4620	1696	36.71
COL	2002/11-2006/7	4455	1361	30.55
JDT	2001/5-2007/12	35386	5089	14.38
MOZ	2000/1-2006/12	98275	5149	5.24
PLA	2001/5-2207/12	64250	9452	14.71
POS	1996/7-2010/05	20431	5119	25.06

B. Data pre-processing

- Delete highly correlated features. REXP is highly correlated with EXP and ND is highly correlated with NF, so REXP and ND are deleted.
- Removal of features used for effort calculation. LA and LD are selected when constructing the effort calculation method and removed when making predictions.
- Data normalization process. LA and LD are normalized by dividing by LT, and LT and NUC are normalized by dividing by NF. Entropy is normalized by dividing by log2(nf).
- Logarithmic transformation. Because most of the features are highly skewed, all features except FIX are processed by logarithmic transformation.
- Handling class imbalance. Most of the data sets used in the experiments are class imbalanced, and the changes without defects are much larger than those with defects. To ensure the reliability of the prediction results,

random undersampling processing is used to deal with class imbalance.

C. Baseline model

In this experiment, we compare EALGB with three models. EALR is an Effort-aware JIT-SDP model based on differential evolution, which is one of the most classical supervised models proposed by Yang et al. [1]. In addition, one supervised model and one unsupervised model, i.e., RF and NF, were selected. We evaluated the models under three different scenarios, namely, 10 times 10-fold cross-validation, cross-project-validation, and time-wise-cross-validation.

The basic principle of 10 times 10-fold cross-validation is as follows: the data set is divided into 10 equal parts, and 9 copies are taken as the training set each time, and 1 copy is taken as the test set. Each piece of data should be used as the test set once. This process is repeated 10 times for a total of 100 validations. Since many early project datasets are too small to form valid datasets, the researchers proposed to use other project data sets to validate the data set of this project. In the second scenario of this article's experiment, five open source projects datasets are used at a time to make predictions for another project, for a total of 30 validations. Considering that randomizing the dataset destroys the original chronological order, it can lead to using future data to predict past defects, thus overestimating the model's predictive performance. Yang et al. [14] proposed a time-wise-cross-validation to address this problem. This scenario is arranged according to the time order of change submission, and the data of each two consecutive months is taken as a group to predict the data of the other two months. In Yang et al. 's paper, the time difference between the training set and the test set is four months, and the same scenario verification is used in this paper. Assuming a project's data set covers n months, the prediction model will produce $n-5$ predictions.

D. Results of the experiment

This paper evaluates the effectiveness of the model in the following three scenarios. The indicators ACC and Popt are used to evaluate the sorting performance of the prediction models. The comparison of experimental results between EALGB and baseline models in terms of sorting performance in the 10 times 10-fold cross-validation is shown in Table 2.

TABLE II. THE SORTING PERFORMANCE IN THE 10 TIMES 10-FOLD CROSS-VALIDATION

Indicator	Project	EALGB	EALR	RF	NF
ACC	BUG	0.606	0.579	0.589	0.467
	COL	0.586	0.446	0.557	0.569
	JDT	0.571	0.560	0.554	0.466
	MOZ	0.542	0.555	0.530	0.322
	PLA	0.622	0.493	0.614	0.469
	POS	0.549	0.531	0.536	0.405
	AVE	0.579	0.527	0.563	0.449
Popt	BUG	0.815	0.707	0.733	0.766
	COL	0.673	0.511	0.646	0.817
	JDT	0.707	0.645	0.665	0.766
	MOZ	0.714	0.631	0.681	0.655



PLA	0.740	0.603	0.710	0.746
POS	0.695	0.659	0.666	0.718
AVE	0.724	0.626	0.684	0.744

Table 3 shows the comparison of experimental results between EALGB and baseline models in terms of sorting performance in cross-project-validation. Table 4 shows the comparison of experimental results between EANDL and baseline models in timewise-cross-validation in terms of sorting performance.

TABLE III. THE SORTING PERFORMANCE IN THE CROSS-PROJECT-VALIDATION

Indicator	Project	EALGB	EALR	RF	NF
ACC	BUG	0.459	0.382	0.400	0.511
	COL	0.476	0.452	0.420	0.469
	JDT	0.661	0.604	0.552	0.469
	MOZ	0.529	0.285	0.437	0.532
	PLA	0.543	0.488	0.534	0.532
	POS	0.710	0.618	0.506	0.494
	AVE	0.570	0.471	0.475	0.513
Popt	BUG	0.800	0.519	0.603	0.772
	COL	0.562	0.576	0.500	0.769
	JDT	0.679	0.743	0.650	0.769
	MOZ	0.555	0.370	0.539	0.772
	PLA	0.675	0.612	0.653	0.769
	POS	0.638	0.718	0.619	0.772
	AVE	0.651	0.590	0.594	0.770

TABLE IV. THE SORTING PERFORMANCE IN THE TIME-WISE-CROSS-VALIDATION

Indicator	Project	EALGB	EALR	RF	NF
ACC	BUG	0.621	0.463	0.450	0.335
	COL	0.418	0.400	0.418	0.420
	JDT	0.449	0.443	0.420	0.392
	MOZ	0.463	0.437	0.451	0.323
	PLA	0.500	0.515	0.500	0.440
	POS	0.463	0.442	0.444	0.390
	AVE	0.485	0.450	0.447	0.383
Popt	BUG	0.856	0.646	0.626	0.682
	COL	0.596	0.553	0.566	0.744
	JDT	0.613	0.579	0.579	0.688
	MOZ	0.640	0.596	0.608	0.651
	PLA	0.641	0.631	0.617	0.734
	POS	0.634	0.597	0.593	0.714
	AVE	0.663	0.600	0.598	0.702

From the experimental results above, we can see that when we use ACC and Popt to evaluate the prediction models, the performance of EALGB is better than the most classical model EALR, supervised model RF and unsupervised model NF in the 10 times 10-fold cross-validation and cross-project-validation scenarios. Table 4 shows that in the time-wise-cross-validation scenario, EALGB is higher than EALR, which means that in the same scenario, EALGB requires significantly less work and still maintains good performance results.

V. CONCLUSION

According to the shortcomings of existing models with unsatisfactory prediction effect and incomplete evaluation, an effort-aware just-in-time software defect prediction model called EALGB is proposed in this paper. And large-scale simulation experiments are conducted from multiple aspects. Experimental results show that using EALGB can effectively improve the accuracy of prediction.

ACKNOWLEDGMENT

This work was supported in part by the the Capacity Building Project of Local Universities Science and Technology Commission of Shanghai Municipality No.22010504100; the Shanghai Rising-Star Program(Sailing Program), China, Grant No.22YF1448100; the Development of Science and Technology of Shanghai Institute of Technology under Grants No.Kjzf2021-176, Kjzf2021-177.

REFERENCES

- [1] Y. Kamei, E. Shihab, B. Adams, et al. "A large-scale empirical study of just-in-time quality assurance," *IEEE Trans. on Software Eng.*, vol. 39(6), pp. 757-773, 2012.
- [2] L. Gong, S. Jiang and L. Jiang. "Research progress of software defect prediction," *J. of Softw.*, vol. 30(10), pp. 3090-3114, 2019.
- [3] X. Chen, Y. Zhao, Q. Wang and Y. Zhidan. "MULTI: Multi-objective effort-aware just-in-time software defect prediction," *Inform. Software Tech.*, vol. 93, pp. 1-13, 2018.
- [4] P. Ardimento, L. Aversano, M. Bernardi, M. Cimitile and M. Iammarino. "Just-in-time software defect prediction using deep temporal convolutional networks," *Neural Comput. Appl.* vol. 34(5), pp. 3981-4001, 2022.
- [5] J. Gesi, J. Li and I. Ahmed. "An empirical examination of the impact of bias on just-in-time defect prediction," *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2021: 1-12.
- [6] Z. Xu, K. Zhao and T. Zhang, et al. "Effort-aware just-in-time bug prediction for mobile apps via cross-triplet deep feature embedding," *IEEE T. Reliab.* vol. 71(1), pp. 204-220, 2021.
- [7] R. Duan, H. Xu, Y. Fan and M. Yan. "The impact of duplicate changes on just-in-time defect prediction," *IEEE T. Reliab.* vol. 71(3), pp. 1294-1308, 2021.
- [8] W. Zheng, T. Shen, X. Chen and D. Peiran. "Interpretability application of the Just-in-Time software defect prediction model," vol. 188, pp. 111245-111236, 2022.
- [9] L. Chen, S. Song and C. Wang. "A Novel Effort Measure Method for Effort-Aware Just-in-Time Software Defect Prediction," *Int. J. Softw. Eng. Know.* vol. 31(08), pp.1145-1169, 2021.
- [10] W. Zheng, T. Shen, X. Chen and P. Deng. "Interpretability application of the Just-in-Time software defect prediction model," *J. Syst. Software.* vol. 188, pp.111245-111260, 2022.
- [11] G. Cabral and L. Minku, "Towards reliable online just-in-time software defect prediction," *IEEE T. Software Eng.* vol. 49(3), pp. 1342-1358, 2022
- [12] J. Carka, M. Esposito and D. Falessi, "On effort-aware metrics for defect prediction," *Empor. Softw. Eng.* vol. 27(6), pp. 152-164, 2022.
- [13] Y. Zhou, Y. Yang and H. Lu et al. "How far we have progressed in the journey? An examination of cross-project defect prediction," *Acm. T. Softw. Eng. Meth.* vol.27(1), pp. 1-51, 2018
- [14] Y. Yang, Y. Zhou, J. Liu, et al. "Effort-aware just-in-time defect prediction: simple unsupervised models could be better than supervised models." *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*. 2016.